Documentación Técnica: Arquitectura de Datos y Pipeline ETL para FleetLogix

Resumen del Proyecto

El objetivo de este proyecto fue diseñar e implementar una arquitectura de datos moderna y escalable para FleetLogix. Pasamos de tener información fragmentada en distintas bases de datos operacionales a un flujo centralizado en la nube.

El pipeline principal está desarrollado en Python y se encarga de unificar dos mundos: los datos transaccionales tradicionales (ventas, rutas) y la telemetría en tiempo real de los vehículos (IoT), para finalmente disponibilizarlos en un Data Warehouse listo para el análisis en herramientas de BI.

Arquitectura y Flujo de Datos

El diseño sigue las mejores prácticas de la industria, separando claramente las zonas de extracción, almacenamiento temporal y carga final.

Orígenes de Datos (Fuentes Híbridas):

PostgreSQL: Actúa como nuestra base de datos relacional (ERP). De acá extraemos los datos estructurados crudos: información de clientes, conductores y el detalle transaccional de las entregas.

MongoDB Atlas: Actúa como nuestro repositorio NoSQL. Implementamos una simulación de dispositivos IoT que inyecta datos semi-estructurados (JSON) con la telemetría GPS de los camiones (coordenadas, velocidad y temperatura del motor).

Capa de Procesamiento (Python & Pandas):

El orquestador principal es un script de Python orientado a objetos (A3-05_etl_pipeline_estudiantes.py).

Usamos pandas para transformar los datos en memoria: limpiamos nulos, cruzamos tablas, calculamos métricas agregadas (como los totales diarios) y "aplanamos" los JSON complejos de MongoDB a un formato tabular.

Data Lake (AWS S3):

Una vez procesados, los datos se empaquetan en archivos CSV (con identificadores de lote o Batch IDs para auditoría) y se envían a un bucket seguro en Amazon S3 usando la librería boto3. Esto nos sirve como zona de aterrizaje (Landing Zone) y resguardo histórico.

Data Warehouse (Snowflake):

El destino final. Python ejecuta comandos COPY INTO de forma remota para que Snowflake absorba masivamente los datos desde AWS S3 hacia las tablas dimensionales (DIM_CUSTOMER, DIM_DRIVER) y las tablas de hechos (FACT_DELIVERIES, FACT_TELEMETRY).

Hitos Técnicos y Desempeño

Durante la ejecución del último lote de producción, el pipeline demostró un rendimiento excelente:

Integración de Datos No Estructurados: Se logró extraer y aplanar con éxito un lote de 10.000 registros de telemetría desde MongoDB, cumpliendo con el requisito de análisis de datos de sensores IoT.

Velocidad de Carga Masiva: Gracias a la estrategia de usar AWS S3 como puente hacia Snowflake (en lugar de hacer inserts fila por fila), logramos cargar un total combinado de 232.867 registros (222k transaccionales + 10k IoT) en aproximadamente 146 segundos, sin registrar errores en la consola.

Idempotencia: El pipeline está diseñado para poder ejecutarse múltiples veces sin duplicar información, manteniendo la integridad del Data Warehouse.

Seguridad y Gobernanza

Un desafío clave de la implementación fue la correcta gestión de accesos en la nube.

Gestión de Credenciales: Todas las credenciales críticas (claves de AWS y la URI de conexión a MongoDB Atlas) fueron extraídas del código fuente e implementadas mediante variables de entorno (archivo .env) para evitar vulnerabilidades de seguridad.

Roles en Snowflake: Se detectó y resolvió un conflicto de gobernanza donde la tabla de telemetría había sido creada por el ACCOUNTADMIN, bloqueando la escritura del script. Se aplicó el principio de mínimo privilegio garantizando que el rol de ejecución (ETL_ROLE) tuviera el OWNERSHIP correcto sobre las tablas del esquema de analítica.

Estimación de Costos Mensuales (AWS Cloud)

Para el cálculo, se asume una operación de 22 días hábiles al mes con una carga masiva diaria y procesamiento de telemetría IoT.

1. Computo: AWS Lambda

- **Uso**: 5 funciones Lambda (extracción, validación y alertas).
- **Cómputo**: 30,000 invocaciones al mes (aprox. 1,000/día).
- **Costo**: El primer millón de solicitudes está incluido en la **Capa Gratuita (Free Tier)**.
- **Total Estimado**: **USD 0.00**

2. Almacenamiento: Amazon S3

- **Uso**: Almacenamiento de archivos CSV crudos (Landing Zone) y backups históricos.
- **Volumen**: ~5GB de datos mensuales (considerando 505k registros + telemetría diaria).
- **Costo**: USD 0.023 por GB.
- **Total Estimado**: **USD 0.12**

3. Base de Datos Administrada: Amazon RDS (PostgreSQL)

- **Uso**: Migración de la base de datos local a una instancia en la nube.
- **Instancia**: db.t3.micro (Single AZ).
- **Costo**: Aprox. USD 0.017 por hora.
- **Total Estimado**: **USD 12.41** (O USD 0.00 si se usa dentro de los 12 meses de Free Tier).

4. Base de Datos NoSQL: DynamoDB

- **Uso**: Almacenamiento del estado actual de las entregas (última milla).
- **Escalabilidad**: On-demand (pago por solicitud).
- **Costo**: USD 1.25 por millón de escrituras.
- **Total Estimado**: **USD 0.50**

5. Monitoreo y Tráfico: API Gateway & CloudWatch

- **Uso**: Punto de entrada para telemetría y dashboard de métricas.
- **Métricas**: 5 métricas clave y 2-3 alertas automáticas.
- **Costo**: USD 1.00 por el dashboard + cargos mínimos por transferencia de datos.
- **Total Estimado**: **USD 1.50**

Resumen Final de Costos

Servicio	Costo Mensual Estmiado
AWS Lambda	USD 0.00 (Free Tier)
Amazon S3	USD 0.12
Amazon RDS	USD 12.41
DynamoDB	USD 0.50
CloudWatch / API Gateway	USD 1.50
TOTAL GENERAL	**USD 14.53**